

Introducing R with behavioural inhibition data

Run R code

In this file, code goes inside chunks like the one below. Click **Knit** to run all the chunks and make an HTML page.

```
4 + 6
```

```
## [1] 10
```

```
(4 + 6) / 2
```

```
## [1] 5
```

You can use the assignment operator to assign values to variables. For instance, we can create a vector of values between 1 and 10.

```
x <- 1:10
```

You can apply any mathematical operation to this vector of integers.

```
x ** 2
```

```
## [1] 1 4 9 16 25 36 49 64 81 100
```

```
sqrt(x)
```

```
## [1] 1.000000 1.414214 1.732051 2.000000 2.236068 2.449490 2.645751 2.828427
```

```
## [9] 3.000000 3.162278
```

Like in Python, you can use `#` to comment things out.

```
# 4 + 6  
# c <- 10
```

Finally, you can also run Python code inside the same environment, which is convenient when you are working on a project that combines both.

```
x = [_ for _ in range(1, 11)]  
print([i ** 2 for i in x])
```

```
## [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Objects, vectors, and functions

`c()` makes a vector. R is case sensitive: `scores` and `Scores` are different names.

```
scores <- c(180, 210, 195, 205, 1) # four SSRT values  
scores
```

```
## [1] 180 210 195 205 1
```

```
length(scores)
```

```
## [1] 5
```

```
mean(scores)
```

```
## [1] 158.2
```

```
round(x=mean(scores), digits=0)
```

```
## [1] 158
```

Try `?mean` in the Console for help.

Use `[ ]` to select values. R starts counting at 1. A comparison returns `TRUE` or `FALSE` for each value.

```
scores[1]
```

```
## [1] 180
```

```
scores > 200
```

```
## [1] FALSE TRUE FALSE TRUE FALSE
```

```
scores[scores > 200]
```

```
## [1] 210 205
```

Numbers, text, and booleans are common types. `class()` shows the type of a variable/value.

```
class(scores)
```

```
## [1] "numeric"
```

```
class("Negative")
```

```
## [1] "character"
```

```
class(T)
```

```
## [1] "logical"
```

NA means a missing value. Many functions need `na.rm = TRUE` to skip missing values (otherwise they return an error).

```
mean(c(180, NA, 205))
```

```
## [1] NA
```

```
mean(c(180, NA, 205), na.rm = TRUE)
```

```
## [1] 192.5
```

Data frames

A data frame has rows for people and columns for variables. `$` selects a column; `[row, column]` selects part of a data frame.

```
people <- data.frame(condition = c("Negative", "Neutral"),  
                     ssrt = c(180, 210))
```

```
people
```

```
##   condition ssrt
```

```
## 1  Negative  180
```

```
## 2   Neutral  210
```

```
people$ssrt
```

```
## [1] 180 210
```

```
people[1, 1]
```

```
## [1] "Negative"
```

Load and check the study data

We are going to look at Paál et al., 2015. In this paper, they were testing a hypothesis that people from more deprived socio-economic backgrounds tend to be more impulsive. To operationalise impulsivity, they relied on the stop-signal task. Participants have to press a key as fast as possible when a certain stimulus appears on the screen. However, if a (stop-)signal is played, the participants need to ignore the stimulus and not press any keys. The duration of the interval between the signal and the stimulus appearance is varied across participants to estimate their inhibition period (how much time they need to stop oneself from responding).

The experiment had 58 participants. Each person completed a stop-signal task after a negative or neutral mood induction. They used mood as a control for testing the effect of the socio-economic background on inhibition. We will look at SSRT (stop-signal reaction time) measures as a function of mood.

Install tidyverse once in the Console with `install.packages("tidyverse")`. Load it in each R session using the `library()` function.

```
library(tidyverse)
```

```
## Warning: package 'readr' was built under R version 4.2.3
```

```
## Warning: package 'dplyr' was built under R version 4.2.3
```

```
d <- read_csv("https://dfzljdn9uc3pi.cloudfront.net/2015/964/1/data_supplement.csv")
names(d)[1] <- "Participant"
```

```
dim(d)
```

```
## [1] 58 13
```

Let's look at the data.

```
head(d)
```

```
## # A tibble: 6 x 13
##   Participant Experimenter Sex Age Deprivation_Rank Deprivation_Score
##   <dbl> <chr> <dbl> <dbl> <dbl> <dbl>
## 1 1 TC 1 65 10137 0.688
## 2 2 TC 2 NA 21612 0.335
## 3 3 TC 1 21 14800 0.544
## 4 4 TC 2 60 2203 0.932
## 5 5 TC 2 42 24295 0.252
## 6 6 TC 2 20 7758 0.761
## # i 7 more variables: Mood_induction_condition <dbl>, Initial_Mood <dbl>,
## #   Final_Mood <dbl>, SSRT <dbl>, GRT <dbl>, Correct_responses_go_trials <dbl>,
## #   Response_Probability <dbl>
```

We can also look just at column names.

```
names(d)
```

```
## [1] "Participant" "Experimenter"
## [3] "Sex" "Age"
## [5] "Deprivation_Rank" "Deprivation_Score"
## [7] "Mood_induction_condition" "Initial_Mood"
## [9] "Final_Mood" "SSRT"
## [11] "GRT" "Correct_responses_go_trials"
```

```
## [13] "Response_Probability"
```

Or look at snippets of the data using the `glimpse()` function from the `dplyr` package.

```
glimpse(d)
```

```
## Rows: 58
## Columns: 13
## $ Participant      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ~
## $ Experimenter     <chr> "TC", "TC", "TC", "TC", "TC", "TC", "TC", ~
## $ Sex              <dbl> 1, 2, 1, 2, 2, 2, 1, 2, 2, 2, 2, 1, 2, 2, ~
## $ Age              <dbl> 65, NA, 21, 60, 42, 20, 64, 31, 20, 20, 20~
## $ Deprivation_Rank <dbl> 10137, 21612, 14800, 2203, 24295, 7758, 16~
## $ Deprivation_Score <dbl> 0.68791946, 0.33464688, 0.54436303, 0.9321~
## $ Mood_induction_condition <dbl> 1, 2, 1, 1, 1, 2, 2, 1, 2, 2, 2, 1, 1, 2, ~
## $ Initial_Mood     <dbl> 100, 60, 70, 100, 95, 85, 100, 85, 90, 90, ~
## $ Final_Mood       <dbl> 100, 50, 70, 100, 95, 90, 100, 80, 90, 85, ~
## $ SSRT             <dbl> 208.0, 222.3, 267.0, 269.0, 170.4, 227.5, ~
## $ GRT              <dbl> 978.5, 547.3, 734.4, 970.1, 905.8, 491.5, ~
## $ Correct_responses_go_trials <dbl> 80.3, 99.3, 100.0, 94.4, 97.9, 99.3, 98.6, ~
## $ Response_Probability <dbl> 46.8, 50.0, 41.3, 42.6, 43.8, 48.9, 46.7, ~
```

And, finally, we can look at some summary statistics for the SSRT.

```
summary(d$SSRT)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 129.9   213.0   234.6   238.4   263.5   392.3
```

The mood codes are 1 for Negative and 2 for Neutral. `mutate()` adds a column.

```
d$Mood_induction_condition
```

```
## [1] 1 2 1 1 1 2 2 1 2 2 2 1 1 2 2 1 1 1 2 2 2 1 2 1 2 1 1 2 1 1 2 2 2 2 1 1 2
## [39] 1 1 2 1 2 2 1 2 1 2 1 1 1 1 2 1 2 1 2 1
```

```
d <- d %>%
  mutate(Condition = factor(Mood_induction_condition,
                           levels = c(1, 2),
                           labels = c("Negative", "Neutral")))
d$Condition
```

```
## [1] Negative Neutral Negative Negative Negative Neutral Neutral Negative
## [9] Neutral Neutral Neutral Negative Negative Neutral Neutral Negative
## [17] Negative Negative Neutral Neutral Neutral Neutral Negative Neutral
## [25] Negative Neutral Negative Negative Neutral Negative Negative Neutral
## [33] Neutral Neutral Neutral Negative Negative Neutral Negative Negative
## [41] Neutral Negative Neutral Neutral Negative Neutral Negative Neutral
## [49] Negative Negative Negative Negative Neutral Negative Neutral Negative
## [57] Neutral Negative
## Levels: Negative Neutral
```

Check the new labels with `count()`.

```
d %>%
  count(Sex)
```

```
## # A tibble: 2 x 2
##   Sex      n
```

```
##    <dbl> <int>
## 1      1     20
## 2      2     38
```

%> is called a pipe. It's implemented in the **magrittr** package, included in **tidyverse**. It passes the result on the left to the function on the right. That way you can combine several operations together without needing to reassign variables or risk losing data.

Practice 1

Can you figure out whether it's a within- or between-participants design? Compare the number of rows with the number of distinct participants. `unique()` returns distinct values, and `length()` counts them.

```
length(unique(d$Participant))
```

```
## [1] 58
```

Describe SSRT

Let's look at mean, median, standard deviation and variance of SSRT. Standard deviation shows spread in milliseconds. `var()` gives sample variance, using $n - 1$.

```
d %>%
  group_by(Sex) %>%
  mutate(mean_age = mean(Age, na.rm = TRUE))

## # A tibble: 58 x 15
## # Groups:   Sex [2]
##   Participant Experimenter Sex Age Deprivation_Rank Deprivation_Score
##   <dbl> <chr> <dbl> <dbl> <dbl> <dbl>
## 1      1 TC      1 65 10137 0.688
## 2      2 TC      2 NA 21612 0.335
## 3      3 TC      1 21 14800 0.544
## 4      4 TC      2 60 2203 0.932
## 5      5 TC      2 42 24295 0.252
## 6      6 TC      2 20 7758 0.761
## 7      7 TC      1 64 16652 0.487
## 8      8 TC      2 31 22105 0.319
## 9      9 TC      2 20 19714 0.393
## 10     10 TC      2 20 26358 0.189
## # i 48 more rows
## # i 9 more variables: Mood_induction_condition <dbl>, Initial_Mood <dbl>,
## #   Final_Mood <dbl>, SSRT <dbl>, GRT <dbl>, Correct_responses_go_trials <dbl>,
## #   Response_Probability <dbl>, Condition <fct>, mean_age <dbl>
```

`group_by()` repeats the summary for each condition. Let's look justn at the mean and standard deviation.

```
d %>%
  group_by(Condition) %>%
  summarise(mean = mean(SSRT, na.rm = TRUE),
            sd = sd(SSRT, na.rm = TRUE))

## # A tibble: 2 x 3
##   Condition mean sd
##   <fct> <dbl> <dbl>
## 1 Negative 242. 50.0
## 2 Neutral 235. 36.8
```

What can you tell about the difference between the two conditions based on these summary statistics?

Practice 2

Does the condition with the higher mean SSRT also have the higher median SSRT? Use `group_by()` and `summarise()` to calculate both for each condition.

```
d %>%  
  group_by(Condition) %>%  
  summarise(median_per_condition = median(SSRT, na.rm=F))
```

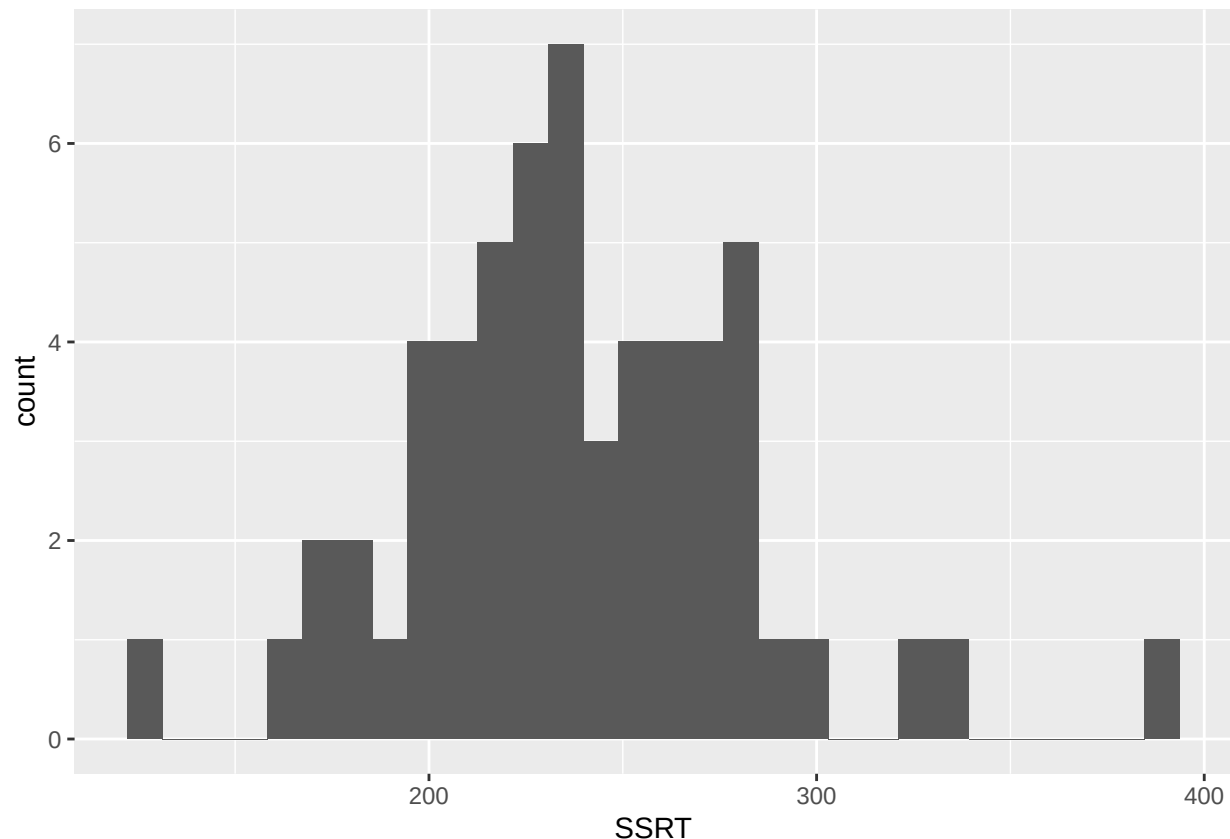
```
## # A tibble: 2 x 2  
##   Condition median_per_condition  
##   <fct>          <dbl>  
## 1 Negative          239.  
## 2 Neutral          234.
```

Plot SSRT

For plotting, we will be using the `ggplot` package. It operates using the “grammar of graphics” system: each new command adds another layer to the initial empty figure instantiated using the `ggplot()` function.

```
ggplot(d, aes(x = SSRT)) +  
  geom_histogram()
```

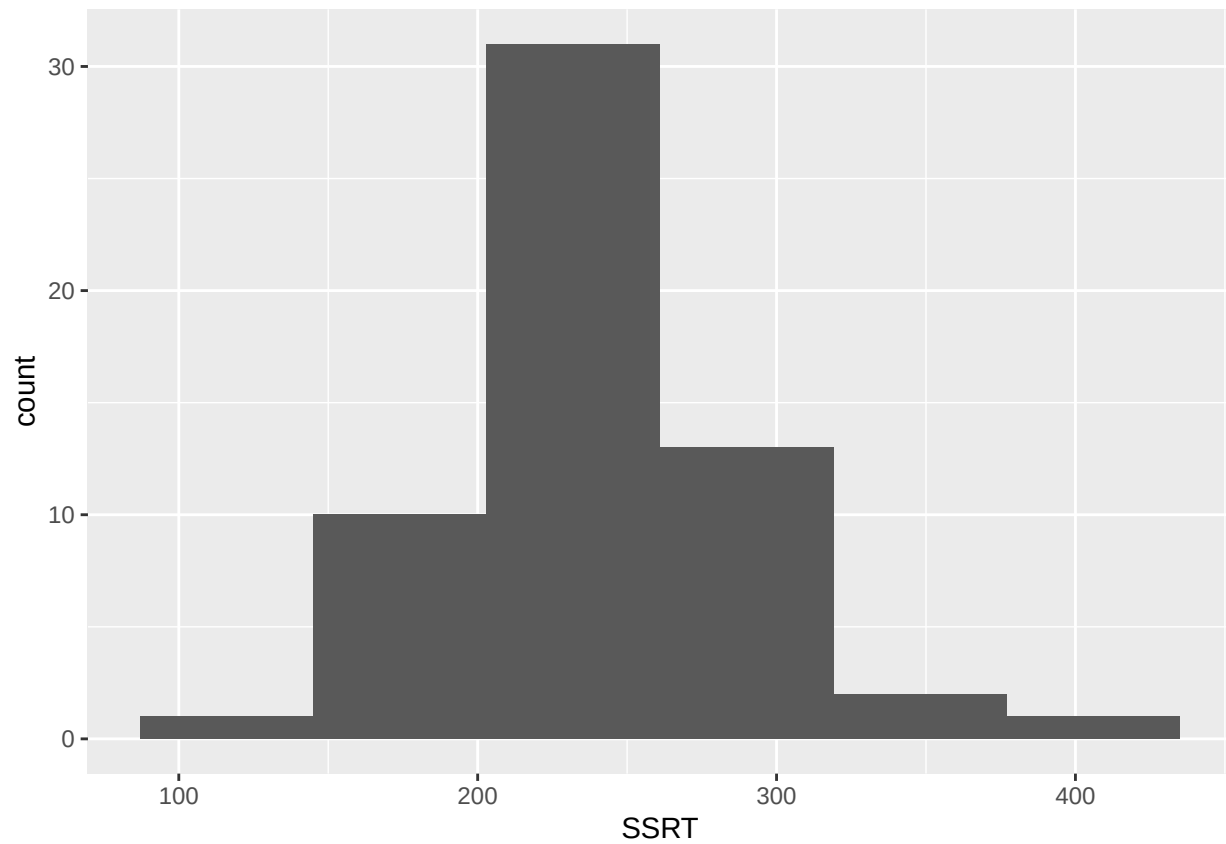
```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



Practice 3

Try making the histogram with `binwidth = 10` and then `binwidth = 40`. How does the choice of bin width change what you notice about the SSRT values?

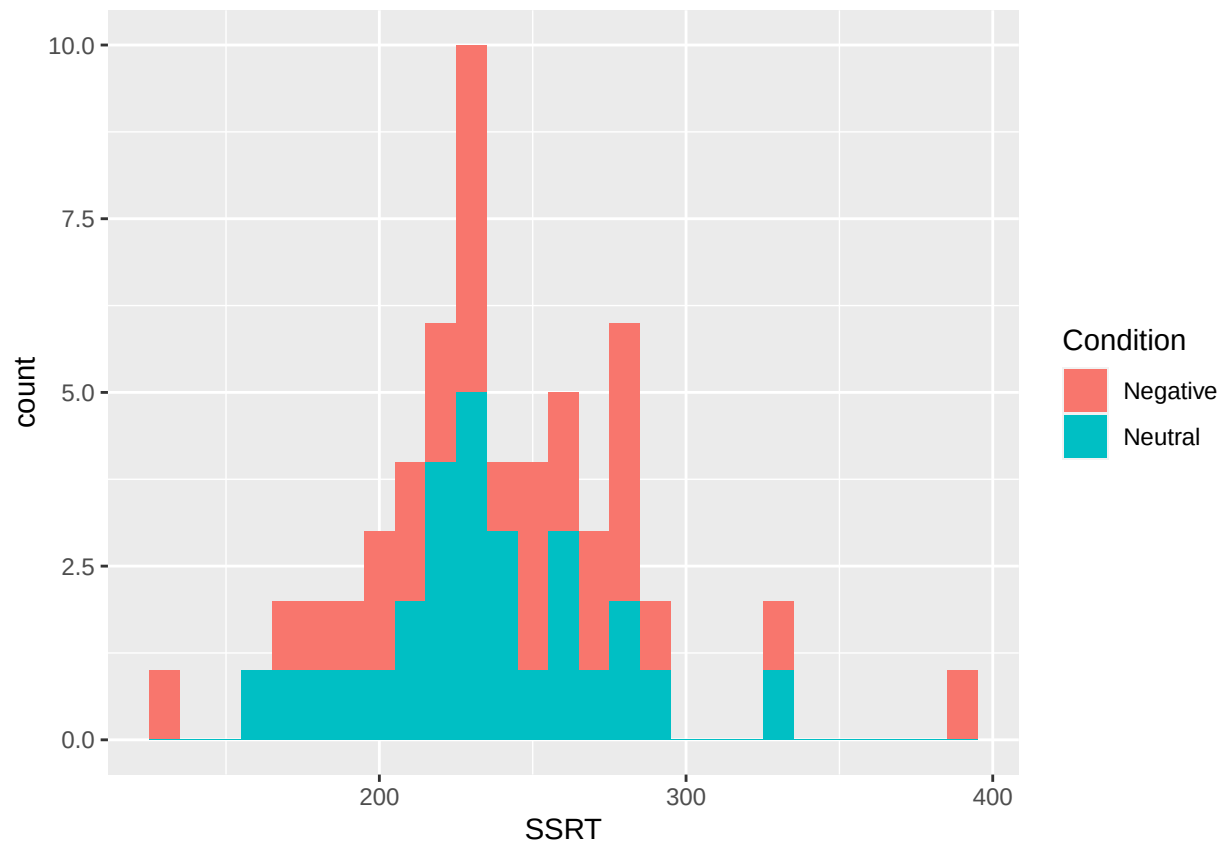
```
ggplot(d, aes(x = SSRT)) +  
  geom_histogram(binwidth = 58)
```



Practice 4

Can you make a separate SSRT histogram for each condition? Do the two distributions look similar?

```
ggplot(d, aes(x = SSRT, fill=Condition)) +  
  geom_histogram(binwidth = 10)
```



Source

Based on Chapters 1–2 of Daniel Nettle, *From Questions to Knowledge: Data Analysis for Psychology and Behavioural Science using R* (2025), online edition. Data from Paál, Carpenter, and Nettle (2015), “Childhood socioeconomic deprivation, but not current mood, is associated with behavioural disinhibition in adults”.